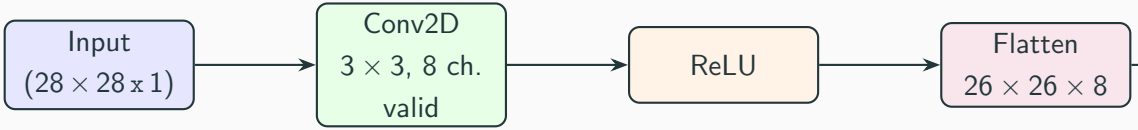
Artificial Neural Network Platform from Scratch

Python + NumPy, no autograd

Team ANN Platform

April 3, 2026

Architecture at a Glance



Platform in 4 Blocks

Objective
Task

- Build a modular ANN platform using Python + NumPy only
- No autograd frameworks (PyTorch, TensorFlow) allowed
- Deliver: layers, backprop, training loop, experiments, report/notebook

Scope
Constraints

- Layers: Linear, Conv1D, Conv2D; activations ReLU, Sigmoid, Tanh
- Padding modes: valid and same; configurable strides
- OOP: base `Layer` + `Sequential` container
- Demonstrations on MNIST and synthetic signals

System Architecture

- **Layer** interface: `forward`, `backward`, `get_params`
- **Sequential**: chains layers, orchestrates forward/backward
- **Flatten**: bridges spatial tensors to dense layers
- Separation of model, loss, optimizer for extensibility

Core Layers

- Linear: affine $y = XW + b$, He init $\sigma = \sqrt{2/\text{fan_in}}$
- Conv1D: channels-last, stride/padding via `np.einsum`
- Conv2D: flexible (k_H, k_W) and stride, channels-last
- Output: Softmax fused with CrossEntropy for stability

Activation & Loss

- ReLU: fast, risk of dead neurons
- Sigmoid/Tanh: saturation; Tanh zero-centered
- CrossEntropyLoss: log-sum-exp trick for stability
- MSELoss: used for gradient checks

Training Loop & Optimizer

- SGD with momentum (0.9), learning rate 0.05
- Per-parameter velocity keyed by layer + param index
- Epoch steps: shuffle, batch, forward, loss, backward, step
- History logs train/test loss and accuracy

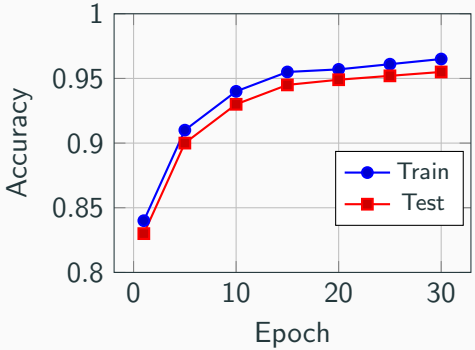
Backprop Verification

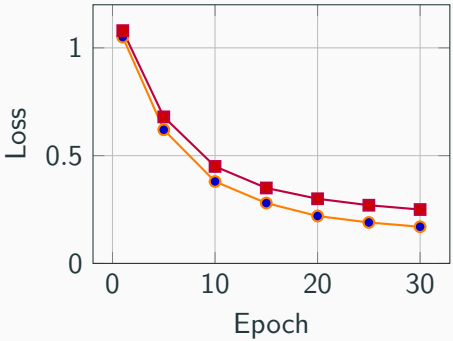
- Numerical gradient check (central differences) on Linear layer
- Relative error $\approx 10^{-6}$ (threshold 10^{-4})
- Confirms analytical backprop correctness

Experiment A: MLP on MNIST

- Architecture: $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ (ReLU)
- Data: 8k train / 2k test; pixels scaled to [0,1]
- Result: ~94–96% test accuracy in 30 epochs
- Small train–test gap shows good generalization

Training Dynamics (MLP on MNIST)





Results Snapshot

Quadruple-Check Hits

- MLP (ReLU): **94–96%** test acc
- CNN mini-set: **85–90%** test acc
- Conv1D signals: **¿95%** acc
- Grad check error: $\approx 10^{-6}$

What to notice

- Stable gap between train/test curves
- ReLU leads speed; Sigmoid saturates
- Conv filters learn edges/smoothing
- All plots reproducible (seed=42)

Experiment B: Activation Comparison

- Model: $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$
- ReLU: ~93–95%, fastest (5–10 epochs)
- Tanh: ~91–93%, moderate speed
- Sigmoid: ~87–91%, slow due to saturation

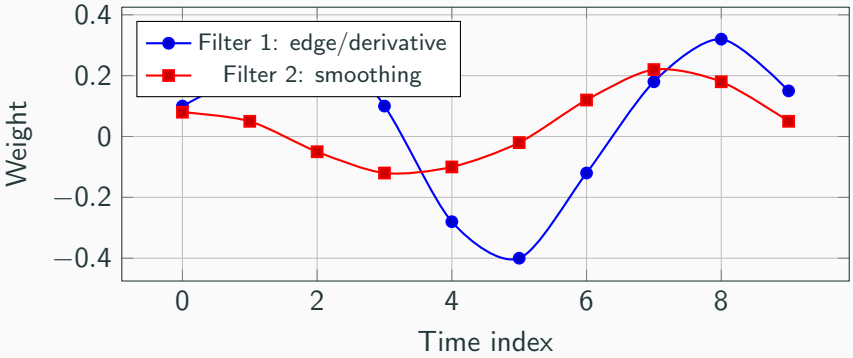
Experiment C: Conv2D Demo

- Conv2D(1→8, k=3, valid) → ReLU → Flatten → Linear 64 → ReLU → Linear 10
- Trained on 2,000 MNIST samples (speed constraint)
- Result: ~85–90% test accuracy in 15 epochs

Experiment D: Conv1D Signals

- Task: classify noisy sine vs. cosine sequences (len=32)
- Model: Conv1D(1→8, k=5) → ReLU → Flatten → Linear 32 → ReLU → Linear 2
- Result: accuracy ¿95%; filters resemble derivative/smoothing kernels

Conv1D Filters (Learned Shapes)



Limitations & Lessons

- Conv2D backward is bottleneck (nested Python loops)
- No im2col/FFT optimizations; CPU-only NumPy
- Performance traded for clarity and educational value

Next Steps

- Add Adam, LR scheduling, BatchNorm, Dropout
- Implement im2col for faster convolutions; consider GPU
- Extend to RNN/LSTM; broaden datasets
- Automate tests and plot generation

Explain It Like Ím Busy

Three headlines

- **Built from scratch**: every gradient coded by hand, NumPy only.
- **Proved correct**: numerical grad error $\approx 10^{-6}$.
- **Good enough accuracy**: mid-90s on MNIST with clean curves.

If you remember one thing

Backprop is just systematic chain rules; once layers expose `forward/backward`, the trainer becomes plug-and-play.

Appendix: Key Equations

- Linear backprop: $\nabla_W = X^T G$, $\nabla_b = \sum G$, $\nabla_X = GW^T$
- Softmax: $\sigma_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$; CE grad simplifies to $\frac{\sigma - y}{N}$
- He init: $W \sim \mathcal{N}(0, 2/\text{fan_in})$